

Proyecto LOST

Learning-based Orientation and Steering for Traversal

Q-Learning y Dyna-Q sobre el ambiente MountainCarContinuous-v0 (Gymnasium)

Alcance. Este informe cubre el Proyecto LOST (primera parte del obligatorio: ambiente Mountain Car Continuous). Se entrega junto con todo el código Python (.py y .ipynb), los modelos computados (.pkl) y los resultados crudos (results/*.csv), de los cuales puede trazarse cada número citado aquí.

1. El problema y su interacción con el simulador

El rover debe aprender, únicamente a partir de la recompensa, que avanzar suele ser mejor que no hacerlo. El ambiente MountainCarContinuous-v0 tiene:

- **Observación continua:** posición $x \in [-1.2, 0.6]$ y velocidad $v \in [-0.07, 0.07]$.
- **Acción continua:** un acelerador $a \in [-1, 1]$ (negativo = empujar a la izquierda).
- **Recompensa:** $-0.1 \cdot a^2$ en cada paso (costo de combustible) más +100 al alcanzar la bandera en $x \geq 0.45$. Los episodios se truncan a los 999 pasos.

La interacción con el simulador sigue el ciclo estándar de Gymnasium: `reset()` entrega la observación inicial (posición uniforme en $[-0.6, -0.4]$, velocidad 0) y `step(a)` devuelve observación, recompensa y las señales `terminated` / `truncated`. El motor es demasiado débil para subir directo: la única solución es *oscilar* para acumular impulso.

Dos propiedades dominan todo el trabajo:

- **La recompensa es dispersa:** una política aleatoria alcanza la bandera en ~2% de los episodios, y Q-Learning de un paso propaga el +100 hacia atrás aproximadamente una celda de la grilla por cada éxito.
- **La recompensa es engañosa:** no hacer nada no cuesta nada. Hasta que el valor de la meta se propaga, "quedarse quieto" es la política que maximiza el retorno: un óptimo local en el que el agente puede quedar atrapado *permanentemente*, porque la política greedy deja de generar la experiencia que lo corregiría.

2. Metodología de evaluación (común a todos los experimentos)

El retorno de entrenamiento es ruidoso y está contaminado por la exploración ϵ -greedy, por lo que toda configuración se evalúa de la misma manera al finalizar su entrenamiento: **100 episodios con la política greedy** (sin exploración) sobre un **conjunto fijo y sembrado de estados iniciales**, idéntico para todas las corridas. Se reporta el retorno medio, su desvío estándar y la **tasa de éxito** (fracción de episodios que alcanzan la bandera). Un episodio exitoso vale aproximadamente +90 (100 menos el combustible); uno fallido, 0 o menos. Cada configuración de los barridos se entrena con **2 semillas** para separar el efecto de un parámetro de la suerte de una corrida. Todos los

experimentos corrieron en un contenedor Linux de 4 núcleos; los tiempos de ejecución citados provienen de `results/*_summary.csv`.

3. Tarea 1 — Discretización de observaciones y acciones

Q-Learning tabular exige conjuntos finitos, así que ambos espacios se mapean a **grillas uniformes** (`discretization.py`): `np.digitize` sobre bordes equiespaciados de posición y velocidad, y `num_actions` valores de acelerador equiespaciados en $[-1, 1]$. La tabla Q tiene forma `(pos_bins+1, vel_bins+1, num_actions)`.

3.1 El conjunto de acciones decide todo

El primer barrido — grillas de estado de 10×10 a 100×100 , todas con **5 acciones**, hiperparámetros base, 3 000 episodios — produjo una pared de ceros: cada corrida, cada semilla, retorno 0.0 y 0% de éxito. La política aprendida es "no hacer nada". El barrido por cantidad de acciones sobre la grilla 20×20 expuso la variable real (`discretization_summary.csv`):

Acciones	¿Incluye acelerador 0?	Retorno eval. (2 semillas)	Éxito
3, 5, 11	sí	0.0 / 0.0	0%
2	no	88.0 / 88.1	100%
4	no	91.9 / 85.0	98–100%

Un conjunto de acciones de tamaño impar contiene una acción "quieto" gratuita que, combinada con el costo de combustible, crea el óptimo local; un conjunto de tamaño par lo elimina estructuralmente: toda política debe moverse, el movimiento funciona además como exploración, y la trampa simplemente no existe. Esta única decisión de discretización logró más que cualquier hiperparámetro (§5). Cuatro acciones superan levemente a dos: los aceleradores $\pm 1/3$ permiten un control más barato cuando basta un empujón suave.

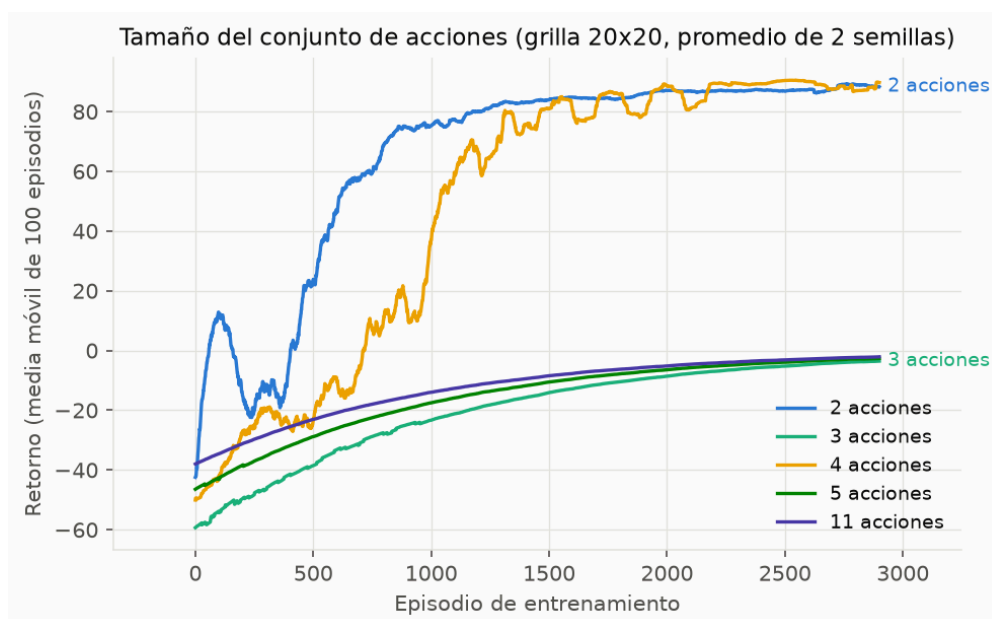


Fig. 2 — Curvas de aprendizaje según el tamaño del conjunto de acciones (grilla 20×20). Los conjuntos impares (3, 5, 11 acciones, con acelerador 0) quedan atrapados en retorno ≈ 0 tras un transitorio; los pares (2 y 4) resuelven la tarea.

3.2 Resolución de la grilla de estados (sobre el conjunto de 4 acciones)

Con 4 acciones el barrido de grillas se vuelve informativo (`discretization_a4_summary.csv`):

Grilla	Estados	Retorno eval. (2 semillas)	Éxito	Pasos promedio
10×10	121	6.0 / 43.2	49–77%	366–617
20×20	441	91.9 / 85.0	98–100%	159–183
50×50	2 601	77.8 / 83.7	84–90%	328–389
100×100	10 201	84.3 / 91.8	92–100%	534–548

El compromiso clásico se ve en ambos extremos: 10×10 sufre *aliasing* (una celda que contiene a la vez "moviéndose rápido a la izquierda" y "casi detenido" recibe un único valor y una única acción), mientras que 50×50 en adelante sigue resolviendo la tarea pero cada celda recibe muchísimos menos datos con el mismo presupuesto, y las políticas resultan más toscas, con trayectorias 2–3 veces más largas. **Discretización elegida: grilla de estados 20×20 × 4 acciones** (441 estados: suficientemente chica para visitarse a fondo, suficientemente fina para separar los estados de impulso que importan).

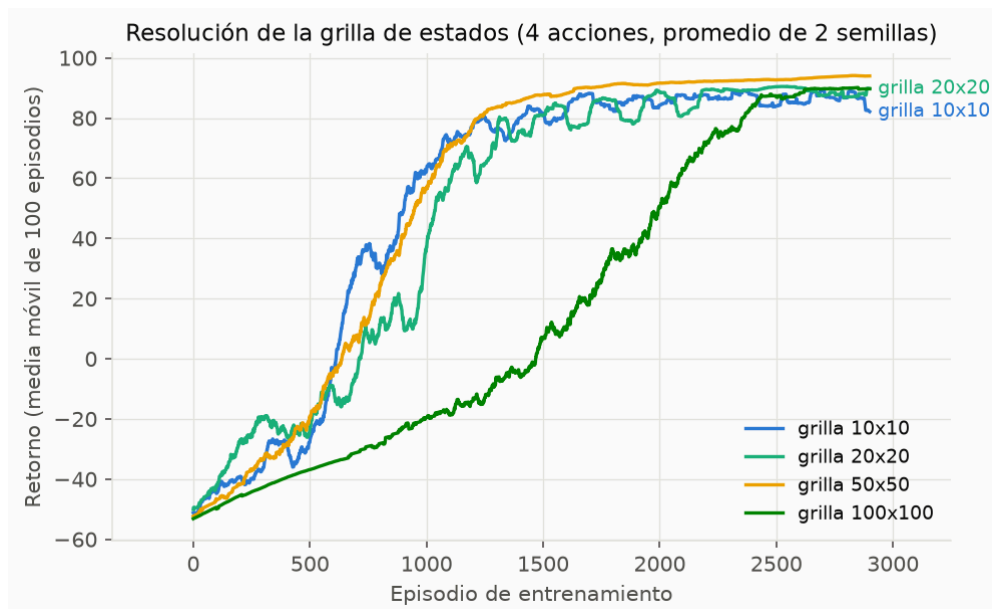


Fig. 1 — Curvas de aprendizaje según la resolución de la grilla de estados (4 acciones). La grilla 100×100 aprende visiblemente más lento por la escasez de datos por celda.

4. Tarea 2 — Técnica: Q-Learning

`q_learning_agent.py` implementa Q-Learning de un paso (control TD off-policy, Sutton & Barto §6.5):

$$Q(S,A) \leftarrow Q(S,A) + \alpha \cdot [R + \gamma \cdot \max_a Q(S',a) - Q(S,A)]$$

con exploración **ϵ -greedy**: ϵ parte de 1.0 (la meta solo puede descubrirse por azar, así que el inicio del entrenamiento debe ser casi totalmente aleatorio) y decae multiplicativamente por episodio hasta un piso `epsilon_min`. En la terminación verdadera (meta alcanzada) se elimina el término de *bootstrap* — el valor de un estado terminal es 0 por definición —; en el truncamiento a 999 pasos se mantiene, porque el límite de tiempo no es un estado terminal real del problema. Los agentes se guardan y cargan como tablas Q serializadas (`models/*.pkl`).

5. Tarea 3 — Exploración de hiperparámetros

Estrategia. Una grilla completa sobre cinco parámetros es inabordable, de modo que se barrió **un parámetro por vez alrededor de una configuración base** (búsqueda por coordenadas): $\alpha \in \{0.05, 0.1^*, 0.2, 0.5\}$, $\gamma \in \{0.9, 0.99^*, 0.999, 1.0\}$, decaimiento de $\epsilon \in \{0.997, 0.999^*, 0.9995\}$ y piso de $\epsilon \in \{0.0, 0.05^*, 0.2\}$ (* = valor base), con 2 semillas por punto, 3 000 episodios, y la evaluación de §2 como criterio de comparación.

Sobre la discretización de 5 acciones (`hyperparams_summary.csv`) el barrido es casi uniformemente cero: ningún valor de α , γ ni del esquema de ϵ escapa de manera confiable de la trampa del "quieto" en 3 000 episodios. La única excepción es reveladora: $\alpha = 0.5$ escapó en una semilla (83.5) y falló en la otra (0.0) — una tasa de aprendizaje grande permite que un único éxito afortunado redirija la política greedy, pero es una moneda al aire, no un método. Una corrida más larga ($\alpha = 0.5$, piso de $\epsilon = 0.2$, 8 000 episodios) sí escapa de forma confiable; ver el tercer modelo final (§7).

Sobre la discretización elegida de 4 acciones (`hyperparams_a4_summary.csv`, Fig. 3) las diferencias se vuelven medibles y consistentes entre semillas:

Parámetro	Valores (retorno medio de eval., 2 semillas)	Lectura
α	0.05: 89.0 · 0.1: 88.5 · 0.2: 92.5 · 0.5: 85.3	0.2 es el mejor; con 0.5 los valores no terminan de asentarse
γ	0.9: 75.4 · 0.99: 88.5 · 0.999: 90.4 · 1.0: 91.3	0.9 vuelve casi invisible el +100 lejano ($0.9^{150} \approx 10^{-7}$); ≥ 0.99 son equivalentes
decaimiento de ϵ	0.997: 90.1 · 0.999: 88.5 · 0.9995: 90.0	despreciable
piso de ϵ	0.0: 88.5 · 0.05: 88.5 · 0.2: 88.0	despreciable: el movimiento forzado ya explora

Elección final: $\alpha = 0.2$, $\gamma = 0.99$, $\epsilon: 1.0 \rightarrow 0.05$ con decaimiento **0.999**, sobre la discretización $20 \times 20 \times 4$ acciones. La lección general: en este problema la *representación* (el conjunto de acciones) importó mucho más que cualquier ajuste fino; los hiperparámetros solo modulan la velocidad una vez que la tarea es resoluble.

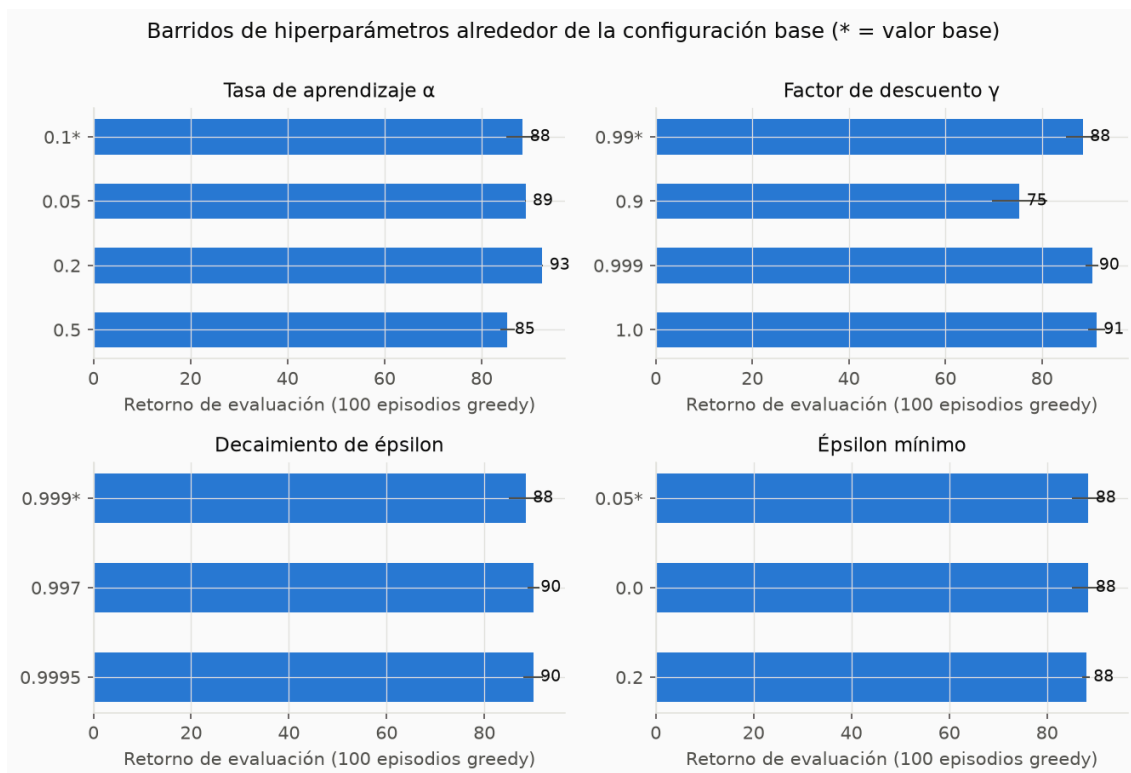


Fig. 3 — Barridos de hiperparámetros alrededor de la configuración base sobre la discretización elegida (barras: retorno medio de evaluación; segmentos: desvío entre semillas).

6. Tarea 4 — Componente de investigación: Dyna-Q (Sutton & Barto, §8.1–8.2)

Las secciones 8.1–8.2 introducen la idea de que un **modelo** del ambiente puede aprenderse junto con la función de valor y usarse para **planificar**: repetir la misma actualización de Q-Learning sobre experiencia simulada. El Dyna-Q tabular (algoritmo del recuadro, p. 164) hace, en cada paso real:

- **(d) RL directo:** la actualización ordinaria de Q-Learning con la transición real.
- **(e) Aprendizaje del modelo:** almacenar $\text{Model}(S, A) \leftarrow (R, S')$. Como la dinámica del Mountain Car es determinista, recordar el último resultado hace al modelo *exacto* para todo par visitado.
- **(f) Planificación:** repetir n veces: muestrear uniformemente un par (S, A) ya visitado, recuperar (R, S') del modelo y aplicarle la misma actualización de Q-Learning.

`dyna_q_agent.py` se implementa como subclase del agente de Q-Learning agregando solo (e) y (f), de modo que la comparación es exactamente uno-a-uno. El experimento da a cada agente el mismo presupuesto reducido de **600 episodios reales sobre la discretización de 5 acciones** — deliberadamente el escenario donde Q-Learning puro queda atrapado — con $n \in \{0, 5, 20, 50\}$ (`dyna_q_summary.csv`, Figs. 4–5):

Agente	Retorno eval. (2 semillas)	Éxito	Tiempo de entrenamiento
Q-Learning / Dyna-Q $n=0$	0.0 / 0.0	0%	10–14 s
Dyna-Q $n=5$	81.6 / 86.1	95–100%	25 s
Dyna-Q $n=20$	55.5 / 61.1	72–86%	58 s
Dyna-Q $n=50$	–27.9 / –27.9	0%	128 s

Tres hallazgos:

1. **$n=0$ reproduce exactamente a Q-Learning puro** — la maquinaria de Dyna en sí no cambia nada; verificación de sanidad superada.
2. **$n=5$ resuelve, en 600 episodios, la configuración en la que Q-Learning puro falló con 5 veces ese presupuesto** — incluso con la acción "quieto" disponible. Es la imagen de §8.2 del libro reproducida en un ambiente más difícil: cada éxito raro es repetido cientos de veces por la planificación, y el +100 barre la tabla en lugar de arrastrarse una celda por éxito.
3. **Más planificación no es mejor: $n=50$ es catastrófico.** La planificación muestrea uniformemente de *toda* la experiencia vista, y antes del primer éxito esa experiencia contiene solo los costos de combustible $-0.1 \cdot a^2$. Con $n=50$ esos costos se martillan en la tabla ~ 50 veces más rápido de lo que el comportamiento ϵ -greedy tarda en tropezar con la meta: todas las acciones de movimiento se vuelven profundamente negativas, el componente greedy deja de moverse, la meta no se encuentra nunca, y la política final (-27.9 en ambas semillas) quema combustible en un vaivén inútil. $n=20$ queda a mitad de camino. Esta es precisamente la debilidad del muestreo uniforme con la que Sutton & Barto motivan el *prioritized sweeping* (§8.4): repetir las transiciones cuyos valores acaban de cambiar, no transiciones al azar.

El tiempo de pared crece aproximadamente lineal con n , como se espera: la planificación compra eficiencia de muestras con cómputo. El tipo de cambio es excelente en $n=5$ ($2.5\times$ el tiempo de Q-Learning puro, mejora infinita del resultado) y ruinoso en $n=50$.

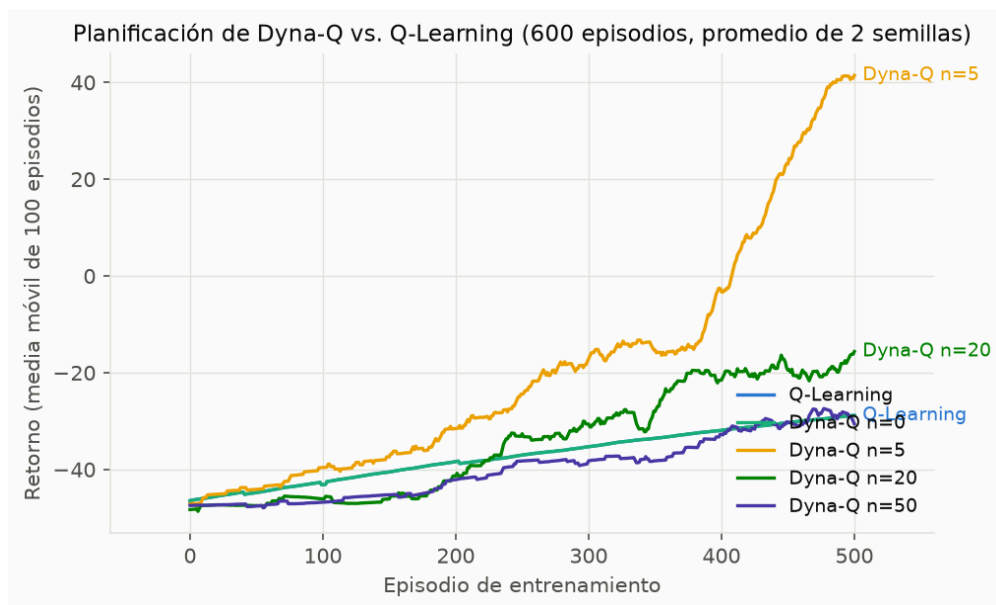


Fig. 4 — Curvas de aprendizaje de Dyna-Q según el presupuesto de planificación, con el mismo presupuesto de 600 episodios reales.

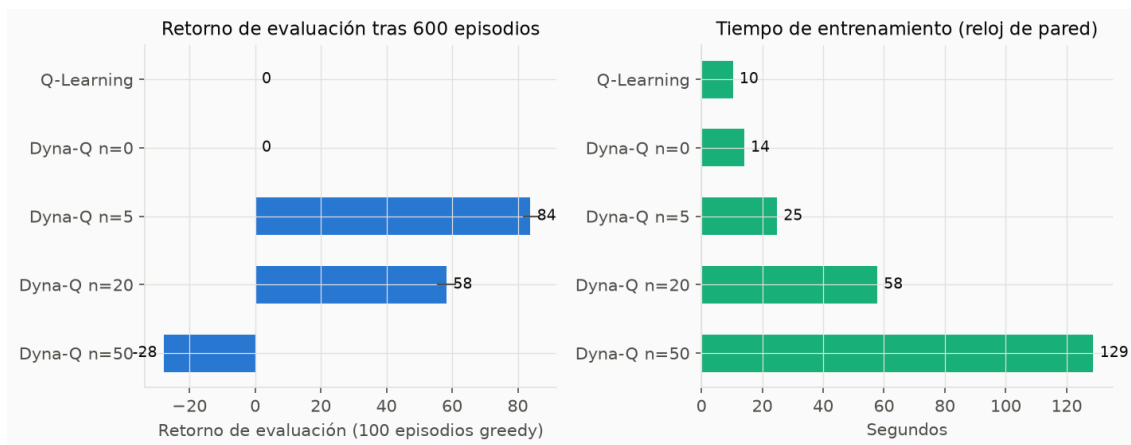


Fig. 5 — Retorno de evaluación y tiempo de entrenamiento según los pasos de planificación: la eficiencia de muestras se paga con cómputo, y el óptimo está en valores bajos de n .

7. Modelos finales entregados y resultados

Se entregan tres modelos computados en `models/` (tablas Q serializadas junto con su discretización), todos entrenados y evaluados con el protocolo de §2 (`final_summary.csv`):

Modelo	Configuración	Episodios	Retorno eval.	Éxito	Tiempo
<code>final_qlearning.pkl</code>	$20 \times 20 \times 4$ acc., $\alpha=0.2$, $\gamma=0.99$	10 000	93.4 ± 0.2	100%	39 s
<code>final_dynaq_n5.pkl</code>	$20 \times 20 \times 5$ acc., $n=5$, $\alpha=0.1$	1 000	77.6 ± 15.8	99%	35 s
<code>final_qlearning_5actions.pkl</code>	$20 \times 20 \times 5$ acc., $\alpha=0.5$, piso ε 0.2	8 000	87.2 ± 26.8	98%	56 s

`final_qlearning.pkl` es **el modelo computado principal de la entrega** (requisito obligatorio del ejercicio 1): verificado además sobre 20 episodios con semillas nunca vistas, obtiene retorno medio 93.3 ± 0.2 , 100% de éxito y soluciones de ~ 78 pasos. El segundo modelo muestra a Dyna-Q rescatando la configuración condenada, y el tercero documenta que la trampa del "quieto" es escapable sin cambiar el conjunto de acciones, pagando α agresivo, exploración sostenida y 8 000 episodios.

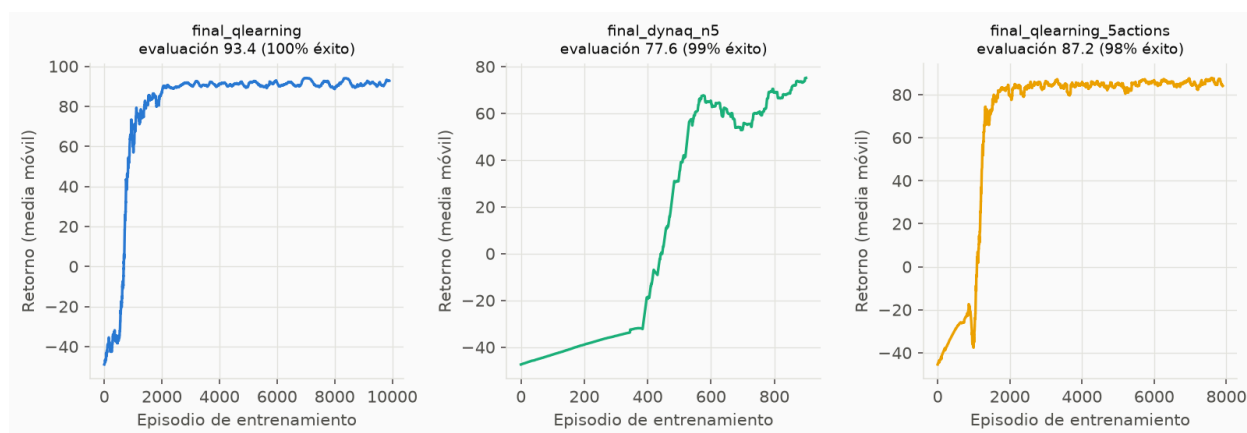


Fig. 6 — Curvas de entrenamiento de los tres modelos finales (media móvil de 100 episodios).

7.1 ¿Qué aprendió efectivamente el agente?

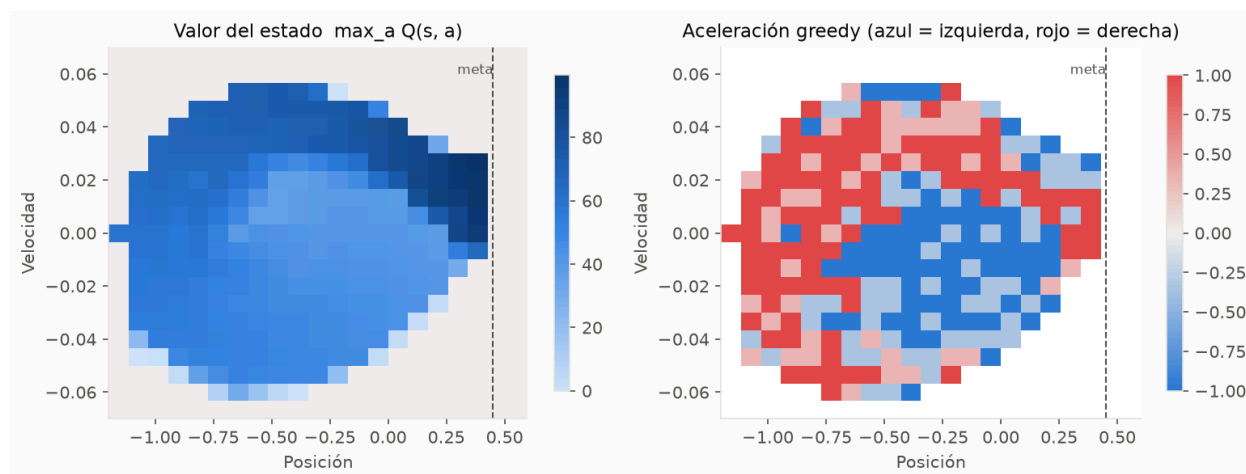


Fig. 7 — Izquierda: función de valor aprendida $\max_a Q(s,a)$; las celdas grises nunca fueron visitadas. Derecha: política greedy (azul = empujar a la izquierda, rojo = a la derecha).

Los dos paneles muestran que el agente entendió la física del problema. El valor es máximo justo a la izquierda de la bandera con velocidad positiva y a lo largo del "corredor de impulso" (el extremo izquierdo con velocidad también es valioso: una pasada por la pendiente izquierda catapulta al auto hacia la meta). La política es un **controlador bang-bang organizado por la velocidad**: moviéndose a la derecha (mitad superior) → empujar a la derecha; moviéndose a la izquierda (mitad inferior) → empujar a la izquierda. Es decir, empujar siempre *a favor* de la velocidad actual para bombear energía a la oscilación — exactamente la estrategia de resonancia que la tarea exige.

8. Conclusiones

1. **La discretización fue la decisión de diseño decisiva, no la regla de aprendizaje.** La paridad del conjunto de acciones esconde una semántica (una acción "quieto" gratuita) que, combinada con el costo de combustible, crea un óptimo local del que Q-Learning tabular no escapa de forma confiable en miles de episodios. Un conjunto par lo elimina estructuralmente; 20×20 equilibra el aliasing contra los datos por celda.
2. **Con la representación correcta, Q-Learning resuelve la tarea con solvencia:** ~ 93 de retorno, 100% de éxito sobre estados iniciales no vistos, soluciones de ~ 78 pasos y una política físicamente interpretable.
3. **Los hiperparámetros importaron menos que la representación:** solo α (mejor 0.2) y γ (necesariamente ≥ 0.99) tuvieron efectos visibles; el esquema de ϵ fue casi irrelevante.
4. **Dyna-Q cumplió su promesa de libro — con la advertencia que el propio libro predice.** $n=5$ pasos de planificación resolvieron con 5 veces menos episodios la configuración en la que Q-Learning puro falla; pero planificar uniformemente sobre experiencia que solo contiene costos es veneno en ambientes de recompensa engañosa ($n=50$ falla por completo), que es exactamente la motivación del prioritized sweeping (§8.4).

9. Notas de advertencia

- Los barridos usaron **2 semillas por configuración** (presupuesto de cómputo): los números individuales cargan algunos puntos de ruido de semilla, aunque todas las conclusiones cualitativas (acciones pares vs. impares, la curva de n en Dyna-Q, el fracaso de $\gamma = 0.9$) son consistentes entre semillas.
- El truncamiento a 999 pasos se trata como **no terminal** en todas las actualizaciones (se mantiene el bootstrap), que es el manejo correcto de los límites de tiempo.
- **No se usó reward shaping en ningún experimento:** todos los resultados son sobre la recompensa propia del ambiente.

- La reproducción completa de los experimentos (`python experiments.py <suite>`) toma ~45 minutos en 4 núcleos; todas las corridas están sembradas.

10. Declaración de uso de Inteligencia Artificial Generativa

Herramienta utilizada: Claude (Anthropic), en su modalidad de asistente de programación (Claude Code).

Contexto de uso: asistencia en la escritura del código de los agentes y del arnés de experimentos a partir de las técnicas especificadas en la letra (Q-Learning según Sutton & Barto §6.5; Dyna-Q según §8.1–8.2), ejecución de los experimentos, generación de los gráficos y redacción inicial del informe. Todos los algoritmos siguen el material citado del curso; todos los resultados provienen de ejecutar realmente el código entregado (los CSV crudos en `results/` permiten verificar cada número), y el contenido fue revisado y verificado contra dichos datos.

Anexo — Estructura del código entregado y reproducción

Archivo	Contenido
<code>discretization.py</code>	Discretizador de grilla uniforme para estados y acciones
<code>q_learning_agent.py</code>	Agente de Q-Learning tabular (entrenar / evaluar / guardar / cargar)
<code>dyna_q_agent.py</code>	Agente Dyna-Q (Q-Learning + modelo aprendido + planificación)
<code>experiments.py</code>	Suites de experimentos: <code>discretization</code> , <code>discretization_a4</code> , <code>hyperparams</code> , <code>hyperparams_a4</code> , <code>dynaq</code> , <code>final</code>
<code>make_plots.py</code>	Regenera todas las figuras (- - <code>lang es</code> para las versiones en español)
<code>continuous_mountain_car.ipynb</code>	Notebook que recorre todo el proyecto, con resultados ejecutados
<code>models/*.pkl</code>	Modelos computados (tablas Q con su discretización)
<code>results/*.csv</code> , <code>*.png</code>	Resumen por corrida, historias por episodio y figuras

El ambiente se gestiona con **Poetry** tal como fue entregado (`poetry install` con el `pyproject.toml` / `poetry.lock` del curso; alternativa: `pip install numpy gymnasium matplotlib`). Para usar un modelo entrenado:

```
agent = QLearningAgent.load("models/final_qlearning.pkl") y luego
agent.test_agent(gym.make("MountainCarContinuous-v0"), episodes=10).
```